

Server — Deep Technical Reference

Written for: you, as the engineer who owns this code. It walks the ASP.NET Core backend **file by file, type by type, method by method**, with the reasoning and the "how it should be done" for each. Companion to [DEVELOPER GUIDE.md](#) (platform/deploy) and [CLIENT INTERNALS.md](#) (the iOS app).

Source: [src/LocationTracker.Api/](#). Stack: ASP.NET Core 9, EF Core 9, PostgreSQL 16, ASP.NET Core Identity, Serilog, behind nginx (TLS) in Docker Compose.

0. Request lifecycle (the 10-second model)

```
HTTPS → nginx (TLS, adds X-Forwarded-*) → Kestrel :8080
  → UseForwardedHeaders          (recover real client IP)
  → ExceptionHandlingMiddleware
  → [Swagger] [HSTS] [static files: /admin/]
  → UseSerilogRequestLogging
  → UseRateLimiter                (per-IP / per-user buckets)
  → UseCors
  → UseAuthentication            (JWT from Bearer header OR a
  → CsrfMiddleware                (double-submit check on cook
  → UseAuthorization            (fallback policy: everything
  → Controllers / /health
```

The order in [Program.cs](#) is load-bearing and commented there; the three that must not move are ForwardedHeaders (before the rate limiter, so buckets key on the real IP), Authentication → Csrf → Authorization

(Csrf needs to know who you are but runs before the endpoint), and static files before auth (the `/admin/` HTML is public; its data is not).

1. `Program.cs` — composition root and pipeline

Top-level statements; there is no `Startup` class. Phases:

1.1 Serilog

`builder.Host.UseSerilog(...)` reads config and writes structured logs to the console (Docker captures them). Request logging is added later with `UseSerilogRequestLogging()`.

1.2 Options binding + fail-fast

- `Configure<JwtOptions>/Configure<TripDetectionOptions>` bind config sections to strongly typed options, injected as `IOptions<T>`.
- **Fail-fast guard:** if `Jwt:SigningKey` is under 32 bytes the app **throws at startup**. A weak HMAC key means forgeable tokens; better to refuse to boot than to run insecure. This is why the README stresses generating a real key.

1.3 Database

`AddDbContext<AppDbContext>(o =>`
`o.UseNpgsql(connectionString))` registers EF Core with the Npgsql provider. `AppDbContext` is scoped (one per request).

1.4 Identity — `AddIdentityCore`, deliberately not `AddIdentity`

AddIdentity registers the Identity.Application **cookie** scheme and makes it the *default* auth scheme, which would silently override JwtBearer. AddIdentityCore<ApplicationUser> gives the UserManager/SignInManager/password hashing without touching authentication. Configured here:

- **Lockout:** 5 failed attempts → 15-minute lockout, enabled for new users.
- **Password policy:** ≥12 chars, upper, lower, digit, symbol.
- **Unique email required.**
- PasswordHasherOptions.IterationCount = 210_000 — raises PBKDF2-HMAC-SHA256 work so a leaked hash is expensive to crack offline.

1.5 Authentication — JWT, read from header or cookie

```
AddAuthentication(default =  
JwtBearer).AddJwtBearer(...):
```

- TokenValidationParameters validate issuer, audience, lifetime, and signature against the symmetric key. **ClockSkew = TimeSpan.Zero** — the default 5-minute grace would extend a 15-minute token to 20.
- JwtBearerEvents.OnMessageReceived — the bridge that makes cookie auth work: if there is no Authorization header, it lifts the token out of the access_token **HttpOnly cookie**. So the same endpoints serve the iOS app (Bearer header) and the admin web page (cookie) with one scheme.

1.6 Authorization — secure by default

```
AddApiAuthorization() (in Security/AuthorizationPolicies.cs):
```

- Policy **AdminOnly** = authenticated + role Admin.
- **Fallback policy = "require authenticated user."** Every endpoint without an explicit [AllowAnonymous] requires auth. A new controller shipped without an [Authorize] is still protected —

you opt *out* of auth, never forget to opt *in*.

1.7 Rate limiting + CORS + forwarded headers

- `AddApiRateLimiting()` — see §7.
- CORS: explicit origin list (`AllowAnyOrigin` is illegal with `AllowCredentials`, and cookie auth needs credentials).
- `ForwardedHeadersOptions` with `KnownNetworks/`
`KnownProxies` **cleared** — otherwise the middleware ignores
nginx's `X-Forwarded-For` and every request looks like it comes
from the container, collapsing all rate-limit buckets into one.

1.8 DI registrations

Scoped: `ITokenService`, `IAuthService`, `ITripDetector`,
`ITripFinalizer`, `ILocationService`. Singleton: `ICookieWriter`.
Hosted: `StaleTripSweeper` (background service). Controllers use a
`JsonStringEnumConverter` so `TripEndReason` serialises as
"`Idle`", not 1.

1.9 Startup work: migrate + seed

In a scope: `db.Database.MigrateAsync()` applies pending EF
migrations, then `IdentitySeeder.SeedAsync` creates the two roles
and the bootstrap admin from `SeedAdmin:Email/Password`. **Caveat
for scale:** migrating at startup assumes a single instance; multiple
replicas would race. Documented as a known limitation.

1.10 The pipeline

As drawn in §0. Static files (`UseDefaultFiles` + `UseStaticFiles`)
serve `wwwroot/admin/`. `app.MapHealthChecks("/health")`. `.AllowAnonymous()` is the container/uptime probe.

`public partial class Program {}` at the end exists only so an
integration-test project can drive the app with

2. Entities (**Entities/**) and the data model

- **ApplicationUser** : **IdentityUser<Guid>** — adds `DisplayName`, `CreatedAtUtc`, and navigation collections. Everything else (password hash, `AccessFailedCount`, `LockoutEnd`, `SecurityStamp`, normalized email) comes from `Identity`.
- **ApplicationRole** : **IdentityRole<Guid>**; `RoleNames.User` / `RoleNames.Admin` are the two seeded roles.
- **Location** — the raw fix: `Id` (long, identity), `UserId`, nullable `TripId`, `lat/lon`, `AccuracyMeters?`, `Speed?`, `Heading?`, `RecordedAtUtc` (device time), `ReceivedAtUtc` (server time, unforgeable).
- **Trip** — the derived journey: start/end coords and times, `DistanceMeters`, `DurationSeconds?`, `PointCount`, `MaxSpeedMps`, `AverageSpeedMps?`, `EndReason?`, `LastMovingAtUtc`, and `IsActive => EndedAtUtc is null`.
- **RefreshToken** — `TokenHash` (never the raw token), `ExpiresAtUtc`, `RevokedAtUtc?`, `ReplacedByTokenHash?`, `CreatedByIp?`, and `IsActive`.
- **TripEndReason** — enum: `Idle`, `ReportingGap`, `Swept` (extend here if you add reasons).

Indexes and integrity (from **Data/Configurations/**)

- `Location (UserId, RecordedAtUtc)` — the workhorse index for "latest fix" and history queries.
- `Location (TripId, RecordedAtUtc)` — for reading a trip's path in order.
- **Trip (UserId) UNIQUE WHERE "EndedAtUtc" IS NULL** — a **partial unique index** that lets the *database* guarantee at

most one open trip per user. This is what makes the concurrency handling in §5 correct rather than hopeful.

- `RefreshToken (TokenHash) UNIQUE`.
- Cascade deletes from user → locations/trips/tokens; `Location.Trip` on-delete `SetNull` (deleting a trip detaches its points, never deletes raw history).

How it should be done: put invariants that must always hold (one active trip) in the database as constraints, not only in C#. Store raw and derived data separately so you can always recompute.

3. Auth stack

3.1 TokenService ([Services/TokenService.cs](#))

- **CreateAccessToken(user, roles)** — builds a JWT with `sub`, `NameIdentifier`, `email`, `name`, a random `jti`, the role claims, and a custom `sstamp` claim carrying the Identity security stamp. Signed HS256 with the configured key; returns token + expiry.
- **CreateSecureRandomToken()** — 48 bytes from `RandomNumberGenerator`, `Base64url` (+/→ -_, no =). URL-safe on purpose: these travel as cookies, and ASP.NET percent-encodes cookie values but not header values; a URL-safe alphabet makes the cookie and the echoed `X-CSRF-Token` header compare equal.
- **Hash(token)** — plain SHA-256. Correct here (unlike passwords) because the input is 48 bytes of CSPRNG output — nothing to brute-force, no slow KDF needed.

3.2 AuthService ([Services/AuthService.cs](#))

The security-critical service. Records `AuthTokens` / `AuthOutcome`.

- `_dummyHash` — a real Identity hash of a throwaway password,

computed once in the constructor. On an unknown-email login the code still verifies against it, so response timing doesn't reveal which emails exist (anti-enumeration).

- **RegisterAsync** — creates the user with `LockoutEnabled = true` (off by default in Identity — easy to miss; without it the failure counter increments but nobody is ever locked). Assigns the role **server-side** (`RoleNames.User`) — never from the request body, or anyone could register as admin. Returns validation errors grouped by code on failure.
- **LoginAsync** — `FindByEmailAsync`; if unknown, burn the dummy hash and return the **generic** failure. Otherwise `CheckPasswordSignInAsync(..., lockoutOnFailure: true)` — Identity drives the brute-force counter and lockout. Locked-out, wrong-password, and unknown-account **all return the same message** — no oracle.
- **RefreshAsync** — hashes the presented token, looks it up. Then, in order:
 1. **Replay detection**: if the stored token is already revoked, someone is replaying a rotated token (stale jar or theft) — **revoke every token for the user** and fail.
 2. Expiry check.
 3. **Security-stamp check**: if the user's current stamp differs from when the token was issued, a "log out everywhere" happened — reject. Otherwise issue new tokens, **rotating** the old one.
- **IssueTokensAsync** — creates the access token, a new refresh raw+hash (storing only the hash), and on rotation marks the old row revoked + `ReplacedByTokenHash`. Returns a fresh CSRF token too.
- **LogoutAsync** — revokes the presented refresh token. **RevokeAllForUserAsync** uses `ExecuteUpdateAsync` (a single SQL UPDATE, no entity loading).

3.3 Cookies, CSRF, JWT options (Security/)

- **CookieWriter** — writes `access_token` and `refresh_token` as `HttpOnly, Secure, SameSite=Strict`; the refresh cookie is scoped to `Path=/api/auth` so it is only sent to auth endpoints. `csrf_token` is **`HttpOnly = false`** (script must

read it for the double-submit).

- **CsrfMiddleware** — skips safe methods (GET/HEAD/OPTIONS/TRACE) and the pre-session login/register paths; only acts when an **auth cookie** is present (bearer clients are exempt — they can't be CSRF'd). Compares the `csrf_token` cookie to the `X-CSRF-Token` header with a **fixed-time** comparison; mismatch → **403**.
- **JwtOptions** — `SigningKey`, `Issuer`, `Audience`, `AccessTokenMinutes` (15), `RefreshTokenDays` (7).

How it should be done: never leak *why* a login failed; make timing uniform; store only hashes of bearer secrets; rotate refresh tokens and treat reuse as compromise; keep CSRF defense for cookie clients and exempt bearer clients.

4. Location ingestion ([Services/LocationService.cs](#))

- **RecordAsync** (single point) and **RecordBatchAsync** (offline backlog) share the shape: load a `DetectionContext`, add each point, run the detector, persist, finalize.
- **LoadContextAsync** — three index-backed reads: the newest stored timestamp (`LatestRecordedAtUtc`), the last minute of *usable* fixes (for the averaged-window detector), and the open trip. Loading once is what lets a 1000-point batch cost the same few reads as one ping. The recent fixes are loaded **tracked** (not `AsNoTracking`) because when a trip opens the detector attaches those rows to it and the change must save.
- **Batch ordering rule** — points are sorted by `RecordedAtUtc` and any point **older than what's already stored is rejected** (`rejected++`), so an offline replay can't rewrite history the detector already acted on. Warnings report the count.
- **PersistAsync / SaveAndDiscardAsync** — wraps `SaveChangesAsync` in a transaction; then calls `TripFinalizer` on the touched trips (closed ones and the still-open one) so the live distance is always the smoothed value; commits.

- **Concurrency (IsActiveTripConflict)** — if two requests race to open a trip, the partial unique index throws 23505. The catch detaches the losing new trip and the FK on its points and re-saves without trip attribution — the point is never lost, only the (re-derivable) trip link.
 - **Reads:** `GetHistoryAsync`, `GetLatestAsync`, `GetLatestForAllUsersAsync` (the admin map's one-row-per-user query, a correlated subquery over the `(UserId, RecordedAtUtc)` index), `GetTripsAsync`, `GetTripDetailAsync` (returns the **smoothed** path, not raw fixes).
 - **ToEntity** normalises every inbound `DateTime` through `.ToUtcKind()` ([DateTimeExtensions](#)) — Npgsql throws if a `timestampz` value isn't `Kind == Utc`, and model binding produces `Unspecified`.
-

5. Trip detection ([Services/TripDetector.cs](#))

Runs synchronously per point inside the ingestion transaction.

- **DetectionContext** — carries `PreviousPoint`, the `RecentPoints` window, `LatestRecordedAtUtc`, the `ActiveTrip`, `ClosedTrips`, and flags. It is what makes batch and single ingestion share one code path.
- **ProcessPoint** — the state machine:
 1. Drop fixes worse than `MaxAccuracyMeters` (100 m) from detection (still stored).
 2. Handle first point / out-of-order / a gap over `GapTimeoutMinutes` (closes the trip with `ReportingGap`).
 3. Reject a segment implying speed over `MaxPlausibleSpeedMps` (70 m/s) as a GPS teleport; after 3 in a row, re-anchor so one bad fix can't wedge detection forever.
 4. **MeasureMovement** — the accuracy fix: compare the

average position over the last 10 s against the average from 20–60 s earlier. Averaging n fixes cuts noise by $\sqrt{n}$. Movement must beat `MovementNoiseFactor` ($2\times$) the combined uncertainty *and* `MinDisplacementMeters` at $\geq$ `MovingSpeedMps`. A good GPS fix (≤ 20 m) with its own Doppler speed is trusted directly.

5. Open a trip **at the baseline point** (so the leg just travelled is included), or extend it, or close it after `IdleTimeoutMinutes` (5) of stillness.

- **CloseTrip** — ends the trip at `LastMovingAtUtc` (not the triggering point), so idle time at the destination isn't counted; computes duration and a provisional average speed.

Distance accumulated here is **provisional** — the finalizer overwrites it.

6. Trip measuring — Kalman smoothing

6.1 `TrackSmoother` ([Common/TrackSmoother.cs](#))

A Kalman filter with an RTS (Rauch–Tung–Striebel) backward pass, per axis, in a local tangent plane (metres east/north of the first fix, so every matrix is 2×2 and cheap).

- Model: constant velocity; state (`position`, `velocity`). Process noise scaled by `SmoothingAccelerationMps2` (0.5) — how hard a phone is expected to accelerate.
- Each fix is weighted by its **reported accuracy** (variance = accuracy^2): vague fixes move the estimate little, precise ones a lot.
- The forward pass filters; the backward pass uses *later* fixes to correct *earlier* ones (a live filter can't). Output: a smoothed track with a per-point speed.
- `DistanceMeters(track)` sums the smoothed path; `ExtentMeters(track)` is the farthest point from the start (used to reject wobble-only "trips").

Why: summing raw hops counts every ± 40 m wobble as travel — the source of the 233 km/h readings. On ± 40 m test tracks the smoothed distance lands within a few percent; raw summing was off 2–15%.

6.2 TripFinalizer ([Services/TripFinalizer.cs](#))

- **FinalizeAsync(trips)** — for each trip: load its usable points ($\leq$ MaxAccuracyMeters, up to LastMovingAtUtc), smooth them, and overwrite DistanceMeters and MaxSpeedMps from the track. For a closed trip, recompute AverageSpeedMps and check IsNoise.
- **IsNoise** — discard if smoothed distance $<$ MinTripDistanceMeters (100 m), or if the track never got TripExtentAccuracyFactor (3 $\times$) its median fix accuracy away from its start. Discard = detach points (raw history kept), delete the trip row.
- **SmoothedPathAsync** — the path the trip-detail endpoint and admin map draw.

This runs inside the ingestion transaction (so figures are always consistent) and from POST `/api/admin/trips/recalculate` for old data.

7. Rate limiting ([Security/RateLimitPolicies.cs](#))

Four partitioned limiters, all keyed on the **real** client IP (recovered by ForwardedHeaders):

- **login** — sliding window, 5/min per IP. Credential stuffing spans many accounts, so per-IP is the layer per-account lockout can't see.
- **register** — fixed window, 3/hour per IP.
- **location-write** — token bucket per **user** (not IP, so users behind one NAT don't starve each other): 120 capacity, +60/min.

Absorbs the burst when a phone reconnects and dumps a batch.

- **Global** — 100/min per IP fallback. `OnRejected` returns 429 with a `Retry-After` header, which the iOS `UploadQueue` honours.
-

8. Controllers (`Controllers/`)

Thin; they translate HTTP to service calls. Every route requires auth via the fallback policy unless marked `[AllowAnonymous]`.

- **AuthController** — `register`, `login`, `refresh`, `logout`, `logout-all` (`UpdateSecurityStampAsync = invalidate every token`), `me`. Writes/clears cookies via `ICookieWriter`.
 - **LocationsController** — `POST /api/locations`, `POST /api/locations/batch`, plus history reads. Role `User` or `Admin`.
 - **TripsController** — `me`, `me/active` (204 when not moving), `me/{id}`.
 - **AdminController** — `[Authorize(Policy = AdminOnly)]`: `users list`, a user's `locations/trips`, any trip, `locations/latest` (the map), `users/{id}/unlock`, and `trips/recalculate[?userId=]`. Asking for another user's trip returns **404**, not **403**, so trip ids can't be probed to learn who exists.
-

9. Background service ([`Services/StaleTripSweeper.cs`](#))

A hosted `BackgroundService` on a `PeriodicTimer` (`SweepIntervalSeconds`, 60 s). Each tick, in a fresh DI scope, it finds trips with no activity past `GapTimeoutMinutes` and closes them (`Swept`), then finalizes. **Required, not optional**: trips otherwise close only when a *later* point arrives, so a phone that dies mid-trip would leave one open forever, blocking the next trip via the partial unique index. A failed sweep is logged and retried next tick, never crashes the host.

10. Configuration surface

([TripDetectionOptions](#))

Bound from the `TripDetection` config section (env vars or `appsettings.json`), tunable without a rebuild:

Key	Default	Meaning
MinDisplacementMeters	15	Jitter floor for a counted move
MovingSpeedMps	1.0	Speed that counts as moving
MaxPlausibleSpeedMps	70	Above this a segment is a GPS teleport
IdleTimeoutMinutes	5	Stillness that closes a trip
GapTimeoutMinutes	15	Silence that closes a trip
MinTripDistanceMeters	100	Shorter trips are discarded
SweepIntervalSeconds	60	Sweeper cadence
MaxBatchSize	1000	Points per batch upload
MaxAccuracyMeters	100	Fixes vaguer than this skip detection/smoothing
MovementNoiseFactor	2.0	Move must beat this × combined accuracy
TripExtentAccuracyFactor	3.0	Trip must reach this × median accuracy from start

JwtOptions: SigningKey, Issuer, Audience, AccessTokenMinutes (15), RefreshTokenDays (7). Cors: AllowedOrigins, Swagger: Enabled, and the ConnectionStrings: Default / SeedAdmin: * values complete the surface.

11. If you extend it — checklist

- **New endpoint?** It's auth-required automatically; add `[AllowAnonymous]` only deliberately. Admin-only → `[Authorize(Policy = AuthorizationPolicies.AdminOnly)]`.
 - **New trip field?** Add to `Trip`, a migration, and set it in the detector/finalizer; expose via the DTO in `Dtos/Trips`.
 - **Tuning detection?** Change `TripDetection` config, not code; re-run `recalculate` for old trips.
 - **New invariant?** Prefer a DB constraint/index over C#-only checks.
 - **Schema change?** `dotnet ef migrations add <Name>` (see the setup guide); migrations apply at startup — for multi-replica, move them to a separate step.
 - **Never** trust client-supplied identity/role; derive `UserId` from the token `(User.FindFirstValue(ClaimTypes.NameIdentifier))`.
-

Keep this in step with the code: when you change a threshold, endpoint, or entity, update the matching section here.